技术方案 · 平台工程

My-Agentix

平台实施方案

第一部分 成本、时延与完成率 • 第二部分 端到端能力与评测补全

项目：My-Agentix 多智能体平台

改进对象：提示词与上下文预算、请求链路、意图路由、MCP、多轮记忆、多智能体分派、在线监控、检索失败处理

口径约定：训练集 / 验证集 / 测试集；分位数为最近邻秩

日期：2026-10-27

目录

摘要 1

1 定位与现状 1

2 提示词版本化管理 4

3 成本压降的四条手段与验证 6

4 链路时延 8

5 任务完成率 10

6 三类指标同表与对外表述 11

7 端到端的意图识别 14

8 MCP 的接入与治理 17

9 多轮对话的记忆 20

10 端到端 Agent 评测 23

11 多 Agent 情况下的路由 26

12 monitor agent 与在线表现 29

13 工具调用与检索失败的优化 32

14 测量基础设施 35

15 施工顺序与工时 36

16 有效性威胁与边界 38

附录 A 未作为主线的部分 39

摘要

这套方案只解决三个可以被追问到底的数字：单位任务的 token 消耗、请求链路的 P95 时延、任务的完成率。主线不是提高某个准确率，而是先把「token 花在哪」变成可归因的账——现有的用量记账只按模型与角色聚合，缺提示词版本这一维，所以任何一次 prompt 改动都无法与 token 变化对上账。方案因此分三步走：先把散在各包里的提示词收敛成带版本号的注册表并把版本号打进轨迹与指标，再在这个前提下做四条压降手段，每一处改动都用同一份回放集验证完成率不掉，最后把三类指标压进一张同表。所有结论的载体是改动前后的对照，不是单次绝对值。

第二部分把范围从指标扩到端到端能力，覆盖七件事：意图识别怎么做、MCP 怎么接进来、多轮对话的记忆怎么管、端到端 agent 怎么评、多智能体情况下怎么路由、monitor agent 的在线表现怎么用、工具调用与检索不全怎么救。这七章共用一套格式——先给代码事实（已经做到什么程度、缺在哪个文件哪一行），再给做法与判据，最后写清这套做法在什么条件下不成立。它与第一部分共享同一套测量底座：轨迹、指标、回放、门禁；第二部分要动的是能力本身，不是另开一套评测。

1 定位与现状

这一章说明为什么把成本、时延、完成率当主线，以及手上已经有什么。地基比预想的厚：分段计时、延迟预算、唯一分位数源、上下文预算与截断、静态前缀保证都已经在代码里，缺的是把它们串成一条能出结论的链。

1.1 为什么主线是这三类指标

三条理由，按重要性排。第一，**样本量决定了准确率类指标做不出可信结论：**

测试集只有十二条，单次准确率的最小可检测差异接近二十八个点，五个点的改进在统计上不可分辨。而 token、时延、完成率都是同一份运行里顺带产生的量，不需要额外标注，也不依赖判分模型。第二，**这三类指标指向的是可复现的工程量**：把预算从两千 token 压到一千二、把路由段从三秒压到一点八秒，换个环境重跑还是这个数；准确率的提升往往换个采样温度就消失。第三，**它们能互相咬住**：单独报「token 降四成」没有意义，砍掉一半上下文也能降四成；必须同时报完成率才能读成一个结论。

相应地，路由准确率与多智能体消融这类指标退到附录。不是它们没有价值，而是它们在本项目的样本规模上无法支撑结论，且与本节要交付的三类指标不共享同一套分母。这个取舍写清楚，比把它们混在同一张表里更诚实。

1.2 已经具备的地基

下面这些能力已经存在，方案不重做，只在它们上面加一列或加一段。这一点很重要：本方案增加的是测量维度与配置面，不是新的运行时。

表 1-1 现有能力与本次要补的位置

能力	位置	本次是否改动
分阶段计时账本	`agentix/performance/latency.py` 的阶段样本与账本，SSE 终帧注入 `breakdown_ms`	补四段
延迟总预算	`agentix/performance/budget.py`，超限只告警并打降级点，fail-open	不改
唯一分位数源	`evals/perf_stats.py` 的最近邻秩实现，含口径标识与切换日常量	加分桶
token 记账	`agentix/observability/metrics.py` 的用量记录，按模型与角色两个标签	加版本标签
上下文预算与截断	`agentix/agent/context/engine.py` 的输入预算与两档确定性裁剪	接版本预算
超长观测隔离	同上，超长内容外置成引用，只留两百字符头部进提示词	不改

能力	位置	本次是否改动
阶梯压缩	agentix/agent/context/compaction.py	不改
静态前缀保证	agentix/domain/policies/prompt_envelope.py 的信封类型与不变性断言	复用
回放与门禁	评测入口的重复执行、基线建档与四态门禁	接版本列

1.3 三个缺口

缺口一：token 没有版本维度

用量记账只有模型与角色两个标签，没有「哪一版提示词」。这不是补一个字段那么简单——没有版本，就没有对照组：改了 prompt 之后 token 涨了百分之十五，说不清是加了三条示例、还是检索多返回了两块、还是模型输出的 JSON 变长了。三个原因混在一个数字里，任何优化都无法判定成败。

缺口二：时延分段缺了最贵的那一段

现有账本只有首 token、检索、流式三段，没有路由、改写、工具与后处理。而路由恰恰是整条链里最慢的一段，占端到端约六成。结果是能说出「路由占六成」，答不出「路由内部模型调用与规则判定各占多少」，也就无法判断该优化规则命中率还是该换更小的分类模型。此外读数没有按路由类型分桶，知识库问答与结构化查询的时延差着数倍，混算的 P95 对任何单一类型都不成立。

缺口三：完成率的分母没写死

任务的成败由状态断言判定，这一点已经做对了——不读模型自述。但剔除项没有约定：超时算不算、环境错误算不算、题面有歧义算不算。这三类每家处理不同，同一份运行能得出相差十来个点的完成率。不把剔除规则写死，完成率就永远是「可以被解释」的数，而不是「可以被比较」的数。

2 提示词版本化管理

这是整套方案的地基。它本身不省一个 token，但没有它，后面四条压降手段只能靠感觉调，也说不清哪一次改动起了作用。

2.1 版本化为什么能省 token

逻辑链很短：**可归因才能做决策**。现在提示词以模块级常量散在四个包里——规划器、护栏前缀、参数提取、会话摘要各一处，外置化和静态前缀排序都做过，但它们是四处独立的常量，没有统一登记，也没有版本号。改一处、跑一次、看到 token 变了，仍然不知道变在哪儿。

Agent 系统里这件事的收益被放大了一层。单轮问答的 token 主要花在一次输入上，改 prompt 是一次性收益；Agent 的每一轮都要重发系统前缀、工具定义、状态栏与历史轨迹，同一个前缀发五次、八次、十次，所以按前缀做一次减法，收益要乘上步数。这也是为什么版本化加预算控制在本项目里比在问答系统里更值得做。

2.2 注册表与五个字段

做法是把四处常量收敛成一张注册表：每个节点一条记录，记录里除模板正文外还有五个字段。注册表本身可以是一个版本化的配置文件，也可以是模块级字典加导入期校验，关键是「一处登记、处处引用」，不允许再有第二份副本。

表 2-1 版本登记要落地的五个字段

字段	作用	缺失后果
版本号	唯一标识一次改写，token 与三项指标的归因键	改动与读数对不上账，无法判定成败
token 预算	该节点输入的上限，超限由上下文引擎确定性截断	预算随内容漂移，同一个版本两次跑的 token 不同
静态前缀声明	供不变性断言校验前缀未被动态内容污染	前缀缓存命中率无声下降，token 反而涨
模型档位	该版本实测所用模型与量化格式	跨档位比较，数字不可复现
指标指针	指向该版本的 token、时延、完成率三读数	报告只能给总数，给不出逐版本对照

版本号建议用「节点名 + 递增序号」，不用日期也不用内容哈希。日期无法表达顺序，内容哈希每次改标点都换版本，反而看不出演进的连续性。真正需要精确指纹时用哈希做二级校验，不作为对外标识。

2.3 版本号怎么进链路

三处写入，一处禁止。**写入轨迹**：每一步记录当前节点的版本号，回放时能还原「这次运行用的是哪一版」。**写入指标标签**：用量记录增加版本标签，token 按节点与版本二维聚合，这是整套方案唯一需要改动的运行时接口。**写入报告**：三项指标同表时，版本号是行标识。

禁止写进静态前缀。这一点最容易被忽略：静态前缀一旦带上版本号或时间戳，每一版都会成为不同的前缀，前缀缓存全部失效，token 不降反升。版本号只落元数据；确实需要模型知道版本时，放在动态后缀，让它出现在前缀之后。这条约束与现有的状态栏白名单是同一个道理——那里只允许常量键，因为每步变化的量会把缓存打穿。

2.4 防回退：冻结、比对、回滚

版本化如果没有回滚能力，只是多了一层记账。三条配套约束：**冻结**——评

测启动时校验模型、温度、采样上限与上下文长度，不一致直接失败，不静默换档；**比对**——每次改版必须跑同一份回放集，把三项指标与上一版并排，只差版本号这一个变量；**回滚**——版本号待在配置里，回滚是改一行配置，不是回退代码。

再加一条自动守卫：前缀稳定性用现成的断言机制跑在测试里，用多组不同输入渲染同一构造器，断言公共前缀长度不低于声明的静态区。它检查的不是代码长什么样，而是输出是否真的不随输入变化——即使有人用条件分支把动态内容混进静态区，只要输出变了，断言就会失败。

3 成本压降的四条手段与验证

有了版本与预算这两个抓手，压降才谈得上方法。四条手段按「收益除风险」排序，每一条都给出验证方式与回退条件。

3.1 四条手段

表 3-1 四条压降手段

手段	做法	收益	风险	验证与回退
上下文预算收紧	检索块按预算截断，每块只留关键句，总预算固定不给浮动	大	低	逐档降预算看完成率拐点，拐点上一档即回退
示例裁剪	少样本示例逐条去掉，掉点即恢复该条	大	中	每条示例单独去掉跑一轮，取最小可用集合
结构化输出	强制固定字段与格式，省掉解释性文字与二次解析重试	中	低	看解析失败率与重试次数，失败率升即回退
历史压缩	多轮只保留摘要加最近若干轮	中	中	按对话轮数分桶看完成率，长依赖任务单独看

顺序有讲究。先做预算收紧与结构化输出这两条低风险的，把 token 打下来一截，同时对完成率影响小；示例裁剪收益最大但最可能掉点，放在有了稳定基线之后做；历史压缩放在最后，它只对多轮任务有效，单轮任务上收益为零。

3.2 验证方法：一次只动一处

四条手段同时上，涨跌无法归因，等于又回到缺口一。正确做法是每次只改一个版本、只跑一项手段，跑完与上一版并排，确认差异超过重复执行的波动再进入下一项。这要求两次运行之间语料、题目、温度、采样上限全部不变，只差提示词版本这一个变量。

验证走回放而不是重新生成：把一次真实运行的模型输出与工具返回录下来，之后所有对比都在回放上跑。这样重跑的成本是机时而不是调用次数，改评分函数、调门禁阈值都不产生新的调用。录制只在语料或提示词大改时重做一次。

3.3 判据：两条曲线，不是单点

单点读数没有说服力，因为总能把预算调到一个看似漂亮的点。要给的是两条曲线：**固定完成率看 token**，即完成率不低于基线的前提下，token 最低能到多少；**固定预算看完成率**，即预算压到某个值时完成率掉多少。两条曲线合起来才说明「这套提示词在这个预算下的真实能力」。

表 3-2 预算档位与完成率的对照表式

输入预算档	单请求 token（中位数）	较基线降幅	完成率	完成率变化	判定
基线档	实测填写	—	实测填写	—	对照
中档	实测填写	实测填写	实测填写	实测填写	降幅最大且完成率不降则采用
紧档	实测填写	实测填写	实测填写	实测填写	完成率跌破基线即为拐点

表里留空是有意的：这三个档位的数值必须跑出来，计划阶段填任何数字都是编。先把表结构定下来，跑完填数，这样报告不会退化成「我们优化了百分之多少」的单句。

3.4 三种不算成果的降幅

三种降幅看起来漂亮但不成立，必须显式排除。**其一，砍到不工作**：把上下文砍到只剩标题，token 降八成，任务也做不成了；所以任何降幅必须配完成率，不配的一律不写。**其二，把成本转到工具调用**：减少模型侧轮次、增加工具侧重试，token 降了但端到端时延和失败率上升，这是把成本挪了位置，不是消掉。**其三，把重试藏进超时**：把超时调短，失败的直接算超时剔除，完成率看起来还升了——这就是为什么剔除项必须在跑之前写死，不能事后补。

4 链路时延

时延的问题不在测不到，而在分段不全与口径不一。这一章先把尺子补齐，再谈优化。

4.1 分段口径

把一次请求切成六段，每段都要有明确的起止与归属。同名阶段累加，段之和与总耗时的偏差控制在百分之五以内，超出说明有未计时的区间或者重复计时。

表 4-1 六段划分与判据

段	覆盖范围	判据	现状
路由	意图分类与规则直判	占端到端约六成，单列看规则命中率	缺
改写	查询改写与同问缓存命中	命中缓存应为毫秒级	缺
检索	向量检索与融合排序	随候选数增长，单独看分位数	有
工具	外部工具调用与重试	含重试，超时单独计数	缺
生成	首 token 到输出结束	与首 token 时延分开记	有
后处理	引用组装、格式校验	应恒定在毫秒级	缺

分段补齐的方式不用改运行时结构：现成的计时上下文在账本未绑定时是零开销的空操作，所以在缺的四段外侧各加一个计时块即可，埋点缺失不会影响主链路。

这一点在实现里已经写死，本方案不需要额外论证安全性。

4.2 三条降延迟措施

路由是主要来源，所以三条措施里两条直接作用于路由。按投入产出排序，前两条改调用路径，第三条只改配置。

表 4-2 三项降延迟措施

措施	做法	预期	风险
规则直判	确定性前缀与正则直接给路由，不进模型	覆盖三到四成问句，省掉整段路由	误判会级联到后续所有步骤，需灰度
同问缓存	路由结果按问题文本哈希缓存，有效期与语料版本绑定	重复问句零路由成本	语料更新后缓存必须失效，否则答旧内容
路由超时收紧	本地慢模型专用的六十秒不用于生产，生产侧按十五秒	尾部时延直接砍掉四十五秒	改错配置会把慢模型整段判超时

第三条有一个具体的坑：本地模型档位下路由超时被抬到六十秒是为了兼容慢速推理，这个值与生产默认值必须独立配置。改之前先确认改的是哪一份，否则会把录制路径整段判超时，录出来的轨迹全是失败样本。

4.3 分位数与冷热启动

三条口径必须统一，否则时延数字之间不可比。第一，**分位数只有一把尺**：全仓使用最近邻秩，入参是零到一的比例而不是百分数，空样本返回空值而不是零——把「没有样本」与「样本都很快」混成同一个返回值，是性能读数最容易自欺的地方。第二，**口径有过切换**：切换日之前的读数与当前不是同一把尺，直接对比会得出反向结论，要做劣化判定必须先按新口径重跑基线。第三，**冷热启动分开**：首次请求包含预热与建连，和稳态请求不是一个分布，混在一起 P95 会被冷启动主导。

还有一条：首 token 时延与全量完成时延必须分列。流式系统里用户感知的是首 token，资源占用看的是总完成；只报一个数，等于选择性地让某一方好看。

4.4 验收判据

时延的验收不是「变快了」，而是三条同时成立：路由段 P95 相对基线下降四成以上；各路由类型分组后每组样本不少于二十条才出 P95，不足则退到中位数；端到端 P95 不低于首 token 时延加生成时长之和的九成五——最后这条是自检，用来发现分段漏计。

5 任务完成率

完成率的难点不在分子，在分母。判定已经是确定性的，缺的是把「算哪些题、剔除哪些题、部分完成怎么记」写成跑之前就固定的规则。

5.1 分母与剔除项

表 5-1 三类剔除项与记录方式

剔除项	为什么剔除	记录方式
环境错误	与被测系统无关，属于题目侧故障	单列计数，不进分母，报告注明
题面歧义	判定标准本身不唯一，不是系统能力问题	单独标注，改题后重新入集
超时	默认**不剔除**，算失败	若因外部服务故障导致，需逐条写明并单列

超时这一行是刻意的：把超时剔除出去，等于允许系统用「拖到超时」换取完成率，而时延恰恰是本文三条主线之一。只有在能证明是外部服务故障时，才允许单列，并且必须逐条写明，不能批量剔除。

5.2 三档判定与拒答

任务结果分三档：完全成功、部分完成、失败。部分完成是否给半分，必须在跑之前声明，并且一旦给分，报告里必须同时出现「完全成功率」，否则与只报完

全成功率的历史读数不可比。

拒答题单独约定两条：该拒而拒算成功，不该拒而拒算失败。这两条要写进断言，不能交给总分口径去处理——把拒答混进普通题里算，会让「多拒」成为一种提升完成率的策略，而多拒恰好会伤害真实可用性。

5.3 稳定性：重复执行的一致性

单次成功率系统性高估可靠性，这是同类系统评测里的已知结论：同一任务重复执行多次，全部成功才算稳定的比例，显著低于单次成功率。因此完成率的报告格式固定为一行三数——单次成功率、重复若干次全对的占比、平均任务成功率——并标注温度与分母。

一条硬约束：重复执行要求采样温度大于零。温度为零时两次运行完全一致，重复指标与单次指标恒等，这个指标就失去区分度。报告必须写明所用温度，否则读数不可解释。

5.4 验收判据

三条：完成率相对基线不降；点估计的下降超过重复执行的标准差才算真下降；三数并列，缺一项则整条结论降级为观察项。

6 三类指标同表与对外表述

这一章把前面三章收口成一张表，并给出对外讲述时必须遵守的规则。它决定这些数字能不能被写进简历还不被追问翻车。

6.1 同表格式

表 6-1 三类指标的口径、采集与目标

指标	口径	采集位置	目标（施工期）
单请求 token	输入加输出，按节点与版本聚合，报中位数与 P95	用量记录增加版本标签	同完成率下降两成以上
链路时延	P95 全量完成与首 token 分列，按路由类型分桶	分段计时补齐六段	路由段 P95 下降四成以上
任务完成率	单次成功率、重复全对占比、平均成功率三数并列，标注温度与分母	评测入口	相对基线不降

表下必须附两行：一行写明模型档位与量化格式，一行写明分母（几条题、剔除几条、为什么）。这两行不是客套，它们决定表里的数字能不能被复现。

6.2 三条纪律

第一，**只写实测值，不写目标值**。上表的目标列是验收条件，跑完之前任何数字都只是计划。第二，**每个数字带分母与档位**，缺一项则该数字不可引用。第三，**降幅必须带对照**：「token 降四成」读起来等于「我把提示词砍了一半」；「token 降四成且完成率不降」才是结论。

这三条同源于一件事：成本类指标天生可以被单方面做好看。所以凡是涉及成本与时延的结论，必须绑定一个不掉的能力指标，否则不成立。

6.3 能写与不能写

表 6-2 表述的边界

能写	不能写	原因
单请求 token 下降若干，任务完成率不降	token 消耗降低八成	未绑完成率的降幅无法排除砍到不工作
路由段 P95 由甲降到乙，端到端 P95 由丙降到丁	系统响应速度提升数倍	未指明段与分位点，无法复现
在固定模型档位与温度下，重复三次全对占比为某值	任务成功率达到某值	缺分母、温度与重复次数，单次值会系统性偏高
提示词按版本登记，改动可归因、可回滚	自研提示词管理平台	如实描述为注册表与配置面，不夸大形态

6.4 第二部分各能力的可写指标

第二部分七章各自能产出可对外表述的数字，但每一项都必须先在基线上跑出来，且必须带口径。下表把这些指标统一收口，避免各章各写一套说法——同一件事在报告里出现两种口径，比不报更糟。

表 6-3 第二部分各能力的可写指标

能力	可写指标	口径与前提
意图识别	路由兜底率、路由段 P95、置信度覆盖率	兜底率须按类别分桶；置信度覆盖率为非空比例，与准确率同报
MCP	接入服务数与工具数、工具调用成功率、降级触发次数	须注明传输方式与超时预算，跨传输的失败率不可直接比
多轮记忆	跨会话注入命中率、长会话单请求 token 中位数降幅	token 降幅必须绑定会话级完成率，否则无法排除「压掉了该记的」
端到端评测	三层用例数、门禁拦截次数、首错定位准确率	用例数要按粒度分列；拦截次数须说明拦截依据
多智能体路由	路由打分 P95、子任务独立完成率、跨智能体信息增量	须声明打开了隔离与契约，默认关闭状态下这些指标无意义
在线监控	低分样本转用例数、兜底告警的提前量	提前量相对端到端成功率下降的时间差，需要至少一个月的连续读数
工具与检索	空结果率、重排后 Recall@5 变化、失败分类覆盖率	Recall 变化须绑定同一份检索金标与同一候选池

这张表有一处刻意的空缺：没有任何一行写「准确率提升多少」。原因是第二

部分要改的是能力结构，它的直接效果是「原来做不到的现在能做到」，而不是「原来做错的现在做对了」——后者的度量需要足够大的分层样本，而第一部分已经论证过现有样本规模撑不起这个结论。所以第二部分的成果应当用覆盖率、成功率、失败率这类计数量表达，不用准确率表达。

7 端到端的意图识别

意图识别在这套系统里不是分类准确率问题，而是「分错了有没有退路」的问题。现状是一次模型调用定生死：判成什么就走什么，判错则整条链路走错，而退路只有一条——落到知识库问答。知识库问答因此成了事实上的垃圾桶类，凡是判不准的都往里丢，但它本身并不保证能答。这一章要改的是退路的结构，不是分类器的精度。

7.1 现状：一次调用，三条兜底，全部落同一个类

入口只有一个函数，做一次结构化输出，返回单个枚举值。类的白名单随开关在七类到九类之间变化，但类型字段始终是单值，没有候选集，也没有多标签。三条兜底路径分别是：模型抛异常、模型返回白名单外的值、状态里根本没有路由结果——三条的落点都是知识库问答，区别只在日志措辞。

规则直判这条快路径已经写好了，但默认关闭。它的判定条件是整句寒暄的精确匹配、法条引用加无推理词、以及长度不超过六十字符三个。问题不在逻辑，在词表：这套正则与关键词是法律业务域留下的，而现在的分类提示词描述的是测试数据域。两边不是同一批词，打开只会误命中，所以线上一直关着。这不是设计缺陷，是域迁移后没对齐，属于必须在启用前先修的前置项。

还有一个不容易注意到的洞：模型返回的对象在进入校验之前有一次类型断言，如果返回的不是预期结构，异常会直接冒泡，既不兜底也不计数。测试把它作为已

知空洞固定下来了，但它意味着「路由失败率」这个指标天然少算一类。

7.2 置信度：先让模型说得出来，再让它说得准

所有关于降级的讨论都卡在同一件事上：现在拿不到可用的置信度。字段在数据结构里有，默认值是空；提示词里还明确要求模型只输出类型字符串、不要添加额外字段。结果是这个字段永远为空。唯一读它的地方是澄清门，而澄清门默认关闭。所以「低置信就走澄清」这条分支在生产上事实上不可达。

修法分两步，顺序不能反。第一步让模型说得出来：提示词放开两个字段，一个是置信度（零到一），一个是判定依据（一句话，说明凭什么归到这一类）；结构校验里把这两个字段设为必填，缺失即视为无效响应并按异常处理。第二步让它说得准：跑一批标注样本，按置信度分十档，统计每档的实际准确率。模型自报零点七绝不等于百分之七十的准确率，弱模型的分布往往整体偏高且挤在零点八到零点九之间。分箱表出来之后，阈值才有依据——取「实际准确率不低于目标线」的最低分档边界作阈值，而不是拍一个零点五。

表 7-1 置信度分箱与动作的对应关系

分档	判据	动作	代价
高	实测准确率高于目标线，且本档样本占比最大	直接分派，不进任何二次确认	无
中	实测准确率落在目标线与可接受线之间	取候选集，前两名都跑，取断言通过者	一次额外链路
低	低于可接受线，或判定依据为空	先澄清；澄清关闭时才降级到最接近的类	一次往返，或一次误判

表里「中档跑两次」这个动作要慎用：它把一次误判的成本换成一次额外的端到端时延，只在时延充裕的路径上开。判据是两条线同时看——路由准确率的提升，与端到端 P95 的抬升，必须同报，只报前者等于把成本藏起来。

7.3 三级路由与降级链

三级的分工不是按准确率分的，是按「错了之后谁来收拾」分的。零级是确定性规则，它不会错，只会漏；一级是单次分类，错了没退路；二级是候选集加验证，允许错但要求能被验出来。三级的触发条件写死在配置里，不靠模型自己判断该用哪级。

降级链要改一个方向：现在的兜底是「一律落知识库问答」，正确的兜底是「落能力最接近的那一个，并把这次降级记下来」。能力接近度靠第 11 章的能力注册表提供，这里只需要约定一件事——任何一次降级都必须留下置信度、依据与目标类别三项，否则线上永远说不清「这个月知识库问答的请求为什么涨了三成」。

这里有一条硬约束：降级不是无限的。同一次请求最多降一级，降完仍失败就返回明确的失败态，而不是继续往下一个类丢。连续降级会掩盖真实故障——一个坏掉的类被反复绕开，表面上成功率还行，实际上那条能力已经死了。

7.4 决策日志与分层评测集

现在路由决策留下的痕迹只有枚举值：一个属性写上类别名，一个计数器记兜底原因，分布进 Redis 的散列表。判定依据这段文字只出现在 HTTP 响应体和流式的一帧元数据里，不落库。这意味着一周后回头看「为什么这条判成了知识库问答」，答案是拿不到的。

表 7-2 路由决策日志的必填字段

字段	取值	用途
类别	有界枚举	分布与分桶指标的唯一维度
置信度	零到一，两位小数	阈值调参与分箱校准的输入
判定依据	单句，限长	事后复盘的唯一可读线索
路由耗时	毫秒，含模型调用与规则判定	补上现有账本缺的那一段
是否降级	布尔，加降级原因	区分「判对了」与「降级后碰对了」

评级集的问题更直接：现有金标全部标注为同一类，一百多条无一例外。这意味着任何把请求判成该类的路由器都能拿到满分，这个指标现在没有区分度。要恢复区分度，必须按类的粒度重标，每类不少于三十条，并且专设一组边界样本——即同时像两类的问题，例如既含法条引用又要求统计的问题。边界样本才是路由器的真实考卷，常规样本只能证明它没瞎。

落库位置上，这五个字段应当写进轨迹而不是只写指标。指标是有界标签的聚合，判定依据属于高基数内容，塞进指标会把标签空间打爆；轨迹才是逐条事后的容器。第 10 章补完轨迹 schema 之后，这一步几乎没有额外成本。

8 MCP 的接入与治理

MCP 在本项目处于一个特殊状态：客户端、服务端、工具源三件代码齐全，重连逻辑有十一个用例覆盖，但生产代码零引用。所有引用都出现在测试与临时脚本里。所以这一章的顺序不是「怎么优化 MCP」，而是「先让它真的被调用，再谈治理」——一个没有流量经过的通道，谈超时、熔断、错误分类都是空转。

8.1 现状盘点

表 8-1 MCP 现有能力与缺口

环节	现状	缺口
传输	仅标准输入输出，子进程加行分帧；协议版本固定为一个旧版本号	无远程传输，无法接入外部服务
重连	最多三次，指数退避，耗尽后进入冷却窗口；有十一个用例	退避无抖动；超时不重连（这条是有意为之）
生命周期	上下文管理器，退出时先关输入再等待，超时杀进程	无空闲回收，无多服务管理
工具暴露	三档工具源：静态、离线、注册表；注册表只导出低风险工具	schema 直搬无适配层；命名靠前缀避让
错误	四类自造异常，另有协议码表	工具不存在被塞进参数错误码；有一个魔法码是字面量

环节	现状	缺口
结果截断	无	大返回直接进上下文
熔断	无，客户端不接断路器注册表	服务挂掉后每个请求都要走完三次重连
生产接线	零	没有服务注册表，没有启动自检

重连这一环是本仓做得最扎实的部分，思路也对：只有传输断流才重连，超时不重连——因为超时往往是服务在忙而不是死了，重连只会加重拥塞。这条约定要在接入时保留，不要为了「看起来更健壮」改成超时也重连。

8.2 接入顺序：四步，前两步不做完不要碰第三步

第一步做服务注册表。每个外部服务一条记录：启动命令与参数、工作目录、环境变量白名单、工具名前缀、默认风险等级、超时预算、是否允许写操作。这张表替代现在硬编码的默认命令，也是「接了几个 MCP 服务」这个数字的唯一来源。风险等级默认给次高级而不是最低——外部工具默认不可信，需要人工把它降到只读档。

第二步做生命周期管理。按需启动、空闲回收，不要一次把所有服务拉起来：每拉一个就是一个常驻子进程，内存与句柄都是成本。启动自检只做一件事——列工具并核对数量，数量与注册表声明不符则告警，不阻断主链路。

第三步做出入口治理，这是接入的真正意义所在。出口把结果送进截断器，入口把错误送进分类器，中间把调用送进熔断器。现在这三样在 MCP 路径上一个都没有：结果原样返回，错误退化成一段自然语言，失败后每个请求都重走三次重连。治理层是现成的，只是没接。

第四步才做传输扩展。本地服务继续用标准输入输出，它的启动成本最低、隔离最好；远程服务加流式 HTTP，因为它是当前协议的主流形态；服务端推送事件那条路线属于过渡态，可以不做。三档传输并存时要统一超时语义，否则同一个工

具在本地三秒失败、远程三十秒失败，问题无法比较。

8.3 错误映射：五类，不是四类

错误分类的价值不在分类本身，在于它决定上层能做什么。上层只有三个动作可选：改选工具、修参数重试、放弃并上报。所以分类的粒度应该按「动作」切，而不是按「异常类型」切。

表 8-2 五类失败与上层的唯一动作

类型	判据	上层动作	是否重试
工具不存在	协议码应为方法未找到，现状错用参数错误	改选工具，查能力表	否
参数不合法	参数校验失败	按错误信息修正一次再调用	一次
传输断流	写入失败或读到流结束	重连后重发	一次
超时	单次读等待超预算	不重试，上报并计入熔断	否
服务降级	重连耗尽，进入冷却	直接失败，冷却期内不再尝试	否

两处具体修法。其一，工具不存在现在复用参数错误码，语义上是错的，会让上层以为「参数写错了，改一下就好」，于是反复改参数重试一个根本不存在的工具。改用方法未找到这个码，上层才能直接换工具。其二，工具级失败现在被折成一个运行时异常，码与是否可重试都丢了，到执行器层只剩下异常类名字符串。修法是保留结构化字段，让执行器能读到码本身，而不是去解析字符串——现在那套字符串关键词二分（超时、限流、连接失败算可重试，鉴权、参数错算不可重试）在中文错误信息上命中率很低。

8.4 超时与熔断的两处硬伤

第一处是超时预算打架。客户端内层固定十五秒，外层工具声明三十秒。内层先超时，外层那三十秒预算永远用不满，等于声明了一个不生效的参数。正确关系是内层取外层的八成，留两成给序列化与回传，并且允许单个工具覆盖。

第二处是退避没有抖动。现在固定零点五、一、二秒。单客户端没问题，但多个客户端同时断连时会以同一相位重连，把刚恢复的服务再打一次。加随机抖动是一行代码的事，改的是「恢复瞬间的雪崩」这类只在生产出现的故障。

熔断接共享注册表之后，冷却窗口才有意义：现在是每个客户端各自冷却，十个客户端就是十份独立的失败预算。接到共享注册表，一个服务挂掉全进程快速失败，恢复也统一判断。

9 多轮对话的记忆

记忆这块的代码量不小：短期有检查点、长期有键值存储、压缩有三层、召回有打分配方、冲突有裁决窗口。但生产上跨会话记忆基本是空的，原因不是设计不行，是写路径断链——摘要的生产者注册好了，却没有一个地方把任务投进去。这一章先补这条断链，再谈打分的精细度。

9.1 三档记忆与各自的生命周期

表 9-1 三档记忆的载体与边界

档	载体	生命周期	内容	现状
工作记忆	检查点存储，按会话标识	七天，读时续期	图状态全量：消息、路由、中间产物	可用，但条数裁到二十条
会话记忆	键值存储，一会话一条	九十天	摘要、主题、关键事实、来源消息区间	读通道通，写通道断
长期画像	键值存储，一用户一条，另加会话有序集合	九十天，每人保留二百条会话索引	用户偏好与稳定属性	结构与整理逻辑齐全，整理默认关

三档的分工要写清楚，否则会出现同一件事在三处各存一份、互相过期。约定是：工作记忆只保证「这一轮说得上话」，不做压缩以外的事；会话记忆保证「明天回来还认得这个会话」，存结论不存过程；长期画像保证「换个会话还认得这个

人」，只存跨会话稳定的偏好。任何一条信息只能有一档主存，其余档只能引用主存的标识。

9.2 写路径断链是最便宜的一处修复

摘要任务的处理函数是完整的：从最近若干条消息里取窗口、调一次摘要模型、脱敏后写入键，同时更新用户会话有序集合并裁剪到二百条。但它没有任何调用方——全仓没有一处把任务投进队列。也就是说，摘要表在真实流量下永远是空的，跨会话记忆这一半功能事实上不存在。

补法有三处触发点，任选其一即可先跑通：会话空闲若干分钟后触发、消息数达到阈值触发、或一轮结束即触发。三者的取舍是成本与新鲜度——每轮都触发会让摘要调用数等于轮次数，成本翻倍；空闲触发最省，但会话残留期间记忆不可用。建议先按「满二十条或空闲十分钟，二者先到」触发，幂等键用会话标识加消息数，避免重复摘要。

这条改造的工作量约半天，是全文性价比最高的一处。它也顺手解决了另一个问题：读通道虽然通，但默认开关是关的，且只在通用问答这一类上注入。也就是说即便摘要有了，也只有一类请求能看到。要把注入点提到路由之后、分派之前，让它对全部类别生效，并保证注入内容不污染检查点——现在它是作为第二条系统消息瞬态注入的，这个做法是对的，要保留。

9.3 召回打分：加两个乘子，不加模型

现有配方是可解释的：关键词命中占一半权重，向量相似度占另一半，融合结果再乘一个时间衰减（半衰期十四天）。有冲突裁决，同键多维冲突时保留置信度最高的那条，并把被裁决掉的值标记在元数据里而不是丢掉。这套设计不比任何黑盒排序差，缺的是两个信号没进公式。

第一个是访问频次。现在命中即自增计数并更新时间戳，但这两个值只写不读，不参与打分。这等于记住了「这条被用过几次」，却不用它来排序。加一个对数平滑的频次乘子即可，不必上复杂的衰减模型。第二个是最近命中时间，与创建时间要分开——一条三个月前创建但昨天刚被用过的记忆，用创建时间衰减会被误判为过期。

有一个坑要提前说：时间衰减的乘子只能重排已有命中，不能创造命中。现在的实现里，融合分小于等于零的候选直接丢弃，这个顺序是对的，改的时候不要倒过来——先衰减再过滤，会让一批本该出局的候选因为衰减系数小而苟活。

表 9-2 记忆侧的四项指标

指标	口径	采集点	用途
注入命中率	被答案实际引用的条数除以注入条数	注入点与答案后处理	判断记忆是有用还是噪声
污染率	注入后被答案否定的条数除以注入条数	同上，配合断言核验	判断长期记忆是否在误导
压缩触发与拒绝	触发次数、通过五要素断言的比例、拒绝原因分布	压缩入口	压缩失败往往是断言太严，需要归因
跨会话召回准确率	给定历史事实，第 N 轮问答能否取回	会话级评测集	唯一能量化记忆效果的指标

前三项是纯统计，不需要标注；第四项必须有会话级评测集，而它现在不存在（见第 10 章）。所以在评测集补齐之前，记忆改造的正确说法是「链路已接通、指标已埋点」，不是「记忆效果提升了多少」。

9.4 压缩：把触发条件从条数换成 token

现在唯一生效的压缩是「丢掉第二十条之前的消息」，按条数算。这一条对短消息有效的场景够用，对长消息完全失效——十条各三千字的消息，裁到二十条等于什么都没做。而真正的压缩器（阶梯压缩加五要素断言）默认是关的，且它的触发条件依赖一个默认为零的 token 预算，两个默认值叠在一起，等于永不触发。

要改的是触发条件，不是压缩算法。把「超过 N 条」换成「超过 token 预

算」，预算按模型上下文的三成给，剩余留给输出与工具返回。压缩动作本身沿用现成的阶梯：先保留最近若干轮原文，把更早的段落压成摘要，压完必须过五要素断言（轨迹标识、错误记录、约束键、来源引用、任务状态），过不了就退到更粗的一档，最粗一档仍过不了则放弃压缩并记一次拒绝——拒绝是正常结果，不是故障。

这里有一个语义陷阱必须点明：上下文引擎的裁剪策略默认值是「从尾部丢」，也就是丢最新一行，这样做是为了保持前缀稳定、让缓存命中；而记忆压缩要的是「从头丢」，即丢最旧的。两者的目的相反。沿用默认值会得到「越新的内容越容易被丢」的荒谬结果，压缩上线的第一个 bug 大概率出在这里。

10 端到端 Agent 评测

现有评测工具链的完成度其实很高：确定性打分、四态门禁、显著性检验、轨迹前缀回归、配对自助法都在。问题出在两个地方——评的对象只有「单轮问答」一种，以及门禁的机位已经撤了（基线目录不再存在，持续集成里只剩单元测试）。所以这一章不是重造评测，是把评测的粒度从一层扩到三层，再把门禁装回去。

10.1 三个粒度，各管一件事

粒度按「一次运行里能被确定地判对错的单位」切，不按模块切。三层各自回答一个不同的问题，混在一起就都答不清楚。

表 10-1 三个评测粒度的分工

粒度	评什么	判分方式	单次耗时	进不进持续集成
单步	路由分类、检索命中、工具选择	确定性断言，与金标严格比对	秒到分钟	进，每次改动都跑
轨迹	工具序列、参数保真、步数效率、首错位置	规则比对，前缀一致加字段一致	分钟	进，仅候选版本
会话	指代消解、约束保持、记忆命中、记忆污染	断言为主，判分模型只做观察	十分钟级	不进，按周跑

会话粒度不进持续集成的理由不是成本，是不稳定：判分模型的方差会直接变成门禁的抖动，一个会随机失败的门禁在两周内就会被绕过。它按周跑、只出报告，报告的用途是发现趋势而不是拦截合并。

10.2 轨迹粒度：现在只测了拓扑，没测过程

现有的轨迹回归比对三个字段：步骤类型、步骤名、状态。这是拓扑漂移检测——它能发现「原来走检索现在不走了」，但发现不了「工具参数从十改成了二十」、「返回内容从三条变成三十条」、「耗时从两百毫秒变成两秒」。这三类恰恰是 Agent 系统里最常见的退化。

数据层还有两个缺口让轨迹级评测做不下去。其一，工具步没有耗时：写入时没传这个字段，恒为零，所以「工具慢了多少」永远算不出来。其二，工具入参与返回不落盘，除非专门打开内容捕获开关，而那个开关默认关闭且只留五百字符。此外仓库里存在两套轨迹结构，一套记步骤与耗时，一套记版本与步骤，各写各的——模型调用只写前者，后者里没有模型步，所以任何一套单独拿出来都无法还原一次完整运行。

修法是合并成一套，字段固定为：序号、类型（模型、工具、节点、检索、错误）、名称、状态、耗时、载荷摘要、token 用量。载荷只存摘要哈希加前若干字符，不存原文——脱敏与体积两个问题一起解决，需要看原文时按哈希回查受控存储。合并之后，四个轨迹指标才有数据基础。

四个指标的定义要说清楚，否则又会变成「跑完看着像那么回事」：**工具序列正确率**指工具调用序列与金标序列的最长公共子序列占比，允许有多余探索但不允许缺关键步骤；**参数保真率**指关键参数（时间范围、实体、过滤条件）与用户意图一致的比例，这一项最能抓「模型自己加了用户没说的条件」；**步数效率**指完成同一任务所用步数相对基线的比值；**首错定位准确率**指第一个失败步骤的判定与人工标注一致的比例，它是所有归因的前提。

10.3 会话粒度：四类用例，每类都要有负例

表 10-2 会话级评测的四类用例

类型	构造方式	通过判据	负例要求
指代消解	第二轮用代词指代第一轮的实体	答案引用的实体与第一轮一致	代词指向歧义时应反问，不得猜
约束保持	第一轮给出过滤条件，第五轮再问	最终答案仍满足该条件	条件被明确撤销后才可放宽
记忆命中	第一轮陈述事实，第 N 轮追问	答案包含该事实且不歪曲	事实是假设语气时不得当成事实
记忆污染	第一轮的临时假设与真实事实并存	后续只引用真实事实	把临时假设带入后续即为失败

四类里最容易漏的是负例。只测「该记住的记住了」，不测「不该记住的没记住」，会诱导系统把整段历史无差别注入——那样记忆命中率确实会涨，代价是污染率同时涨，而后者在真实使用里更致命。所以每一类都必须配一条反向用例，且反向用例算在同一个分母里，不允许单独成表。

会话级用例的构造成本是这套方案里最高的一项。一个可行的降本方式是用真实会话改造：从生产会话里挑出含有明确指代与明确约束的片段，人工改写为问题加断言，一条会话能拆出三到五条用例。这种方式比从零手写快一个数量级，代价是覆盖面偏向常见场景。

10.4 判分与门禁

判分顺序固定为三层，越靠前越优先：第一层是字符串与结构断言（必须包含、必须不包含、字段是否存在、数值是否在区间），第二层是规则断言（引用是否可追溯、证据是否支撑、参数是否越界），第三层才是模型判分。前两层能判的绝不进第三层，因为第三层既贵又不可复现。现有的相似度判分已经在观察位而不是门禁位，这个定位要保留。

门禁要装回去三件事。第一件是有基线：现在基线目录已经不存在，四态门禁里「未武装」那一态长期成立，等于门禁开着但不拦人。第二件是有任务：持续集成里要有一个独立作业跑单步与轨迹两层，失败即阻断，而不是并入单元测试。第三件是基线带版本列：第 2 章的提示词版本号要成为基线的行标识，否则每版都覆盖上一版，「这一版比上一版差多少」这个问题永远问不出来。

有一条纪律要写进门禁脚本本身：基线只允许从唯一通道建档，且建档时拒绝空键。现在这套约束已经实现了（零值键不写、缺预测拒绝建档），是全仓少见的防自欺设计，扩写时不要绕过它。

11 多 Agent 情况下的路由

先说一个必须承认的事实：现在的「多智能体」名不副实。整条链路只执行一个处理函数，所谓的提议者就是那个函数的草稿，审阅者拿同一份草稿做结构检查。这不是多智能体协作，是单智能体加自审。所以附录里那组多智能体消融测到的差异，本质是「自审有没有用」，而不是「分工有没有用」。这一章讨论的是真做多智能体时，路由该怎么设计。

11.1 现状：静态查表，无元数据，无仲裁

分派是一张写死的字典：八个类别名对应八个函数，查不到就抛异常。这张表没有描述、没有成本、没有健康度、没有输入输出契约，所以它无法支撑任何「挑一个更合适的」决策，只能支撑「按名字取一个」。

能力声明表其实已经写了一份，按角色列出可用工具集与上下文策略，但全仓没有一处引用它——声明与实现脱节。目标仲裁的逻辑更完整：六条硬优先级、八组冲突对的确定性裁决、以及「安全不可被配置让路」这条底线，全部实现并可测，同样零接线。这两份资产是这一章能快速推进的前提，不需要从零设计。

另外三个开关——消息总线、结果契约、状态隔离——默认全关。默认路径下，父级的完整轨迹与活动记忆原样进子智能体，子智能体的失败也会直接污染父级预算。单智能体时这没问题，真上多智能体时必须先打开隔离。

11.2 能力注册表：路由的候选集从哪来

现在的候选集是硬编码的八个名字，未来的候选集必须来自注册表。表里每个智能体一条记录，字段不多，但每个字段都要能被程序读、不能只写给人看。

表 11-1 能力注册表的字段

字段	内容	被谁消费
能力标识与描述	稳定标识，加一句自然语言描述用途与边界	分类器做语义匹配
输入输出契约	接受的参数类型、返回的结构（状态、论断、证据引用、成本）	调用方校验，编排层聚合
成本画像	单次调用的 token 与工具调用数中位数	路由打分里的成本惩罚
时延画像	P50 与 P95，分输入规模	路由打分里的时延惩罚
健康度	近若干次调用的成功率与失败模式分布	降级与熔断判断
依赖	依赖的工具、模型档位、外部服务	启动自检与影响面分析

其中「描述」这一项最容易被写成空话，但它恰恰是分类器做语义匹配的唯一

依据。写法上要求把它写成「什么时候该用我、什么时候不该用我」，而不是「我是一个专业的检索助手」。后者对匹配毫无信息量。

11.3 从查表到打分：四个因子，一个阈值

候选集有了，分派就从「取一个」变成「排序后取第一个，并检查领先幅度」。四个因子相乘或加权，每一项都要能在日志里单独看到，否则调参就是盲调。

表 11-2 路由打分的四个因子

因子	来源	作用方向	失控时的表现
能力匹配度	分类器对该类别的概率	越高越优先	分类器偏科，某类永远赢
历史成功率	该智能体在本类任务上的近 N 次成功率	越高越优先	强者恒强，新智能体永不获客
成本惩罚	token 与调用数中位数	越高越靠后	成本权重过大，全部挤向最便宜那个
时延惩罚	P95 相对预算的占比	越高越靠后	时延权重过大，长任务被推给弱智能体

阈值指「第一名与第二名的分差」。分差足够大就直接派第一名；分差小于阈值说明两个能力都说得通，此时有两条路：串行备选（先跑第一名，断言不过再跑第二名）或并行竞速（两个都跑，取先通过的）。串行省成本但慢一倍，并行快但成本翻倍，选择依据是这条路径的时延预算是紧还是松。

四个权重不能一次定死，要用第 20 章的轨迹评测集做网格搜索，且搜索的约束是「时延不超预算」而不是「准确率最高」。理由是准确率可以靠并行竞速无脑抬高，那不叫设计好，叫花钱买分。这一点必须在报告里说清楚，否则读者会以为分数低是能力差。

11.4 隔离、契约与灰度

三个开关要按顺序打开，不能一起上。先开隔离：子智能体用临时提示、跳过记忆、只读快照、消息面只留最后一条。这一步不改任何业务逻辑，只堵住父级上

下文泄漏。再开契约：子智能体返回结构化的结果（状态、论断、证据引用、置信度、成本），而不是一段自然语言。有了契约，编排层才能判断「证据够了没有」并提前收敛，也才能把多个子结果合并而不靠字符串拼接。最后才开总线：进程内队列的收益只在真正并发时体现，单机串行场景下它是纯开销。

多智能体才有意义的两个指标，单智能体下测不出来：**子任务独立完成率**（去掉父级上下文后，子智能体单独能不能完成）与**跨智能体信息增量**（后一个的产出相对前一个有多少新信息）。信息增量的判据代码里已经有了，按新词占比算，重叠度超过阈值视为无增量。这两个指标一起看才说明分工是否成立：独立完成率低说明子智能体只是在蹭父级上下文；信息增量接近零说明多个智能体在重复劳动。

灰度现在是个空壳：晋升接口收一个百分比参数，把它塞进负载就不再管；也没有任何代码去读取提案并应用。真做灰度要按会话标识哈希分流，并且要求同一会话固定落在同一侧——否则用户会看到同一个问题两次给出不同口径的答案，那比不做灰度更糟。

12 monitor agent 与在线表现

先厘清一个定义问题。monitor agent 的价值不在「再调一次模型看答案对不对」——那个做法成本高、与被测系统同源、结论不可复现，属于用模型评判模型的循环论证。它真正的价值是「读确定性信号、定位第一个出错的环节、产出一个可执行的动作」。所以它应该是规则引擎加归因表，而不是另一个提示词。

12.1 现状：观察者组件是死代码，真正的监控在别处

仓库里那个叫 observer 的组件只有二十几行，是一个内存里的追加列表，没有落盘、没有阈值、没有回调，全仓没有一处调用它。真正的在线监控分散在四个

地方：指标库（约四十个指标）、告警规则（八条）、降载阶梯（七级）、熔断器。这四样都在，问题是它们之间不闭环——告警响了，没有人把它变成一条可复现的用例。

指标本身也有两个硬缺口。其一，用量记录的请求计数把状态硬编码成成功，模型调用失败根本不计数，于是「失败率」这个指标不存在。其二，没有记忆、压缩、检查点恢复相关的任何指标，第 9 章那些改造上线后无法观测。这两处不补，monitor agent 的输入就是残缺的。

12.2 三类信号与各自的动作

表 12-1 三类在线信号与对应动作

类别	信号	阈值来源	动作
质量	锚定低分率、引用不可验数、证据门禁触发次数	按周基线的分位数，不拍绝对值	追加警告标记，抽检进候选用例
效率	路由段 P95、端到端 P95、单请求 token 中位数	延迟总预算与上一周基线	超预算记降级点，切规则直判
稳定	六类协作失败计数、路由兜底次数、工具错误分类计数、服务降级次数	绝对阈值加环比	熔断、降载、切备用模型

质量类的阈值要用分位数而不是绝对值，原因是锚定分数依赖证据池与答案长度，绝对值在不同语料上不可比。这一点和时延指标是同一个道理，也是为什么两者必须共用一把分位数尺。

稳定类里有一个值得单独说的信号：路由兜底次数。它上涨通常意味着上游提示词或模型档位变了，而不是路由本身坏了。这个信号的价值是「最早的告警」——它比端到端成功率下降早出现，因为兜底到知识库问答往往还能给个答案，只是答得不对。把它放进告警规则，等于给整条链路装了一个前哨。

12.3 信号回灌的三条通道

第一条是告警，已经实现，八条规则覆盖服务存活、错误率、时延、限流、降级、熔断、锚定低分、模型兜底。缺的是质量类规则只有一条，要把引用不可验与证据门禁触发补上。**第二条是降级**，七级阶梯已经写好但默认关闭，接上之后要注意方向：质量型降级应该收检索参数（缩小候选池、提高阈值），而不是直接换小模型——换模型的代价是能力断崖，收参数的代价只是可能答得更保守。**第三条是数据回灌**，这是最有价值的一条：把线上低分样本转成候选用例。

回灌通道其实已经有了：点踩事件可以导出为候选金标行，带上错误码与待复核标记，且明确不自动改写金标。这条设计的克制是对的，要保留。需要补的是自动触发——现在需要人工导出，应该改成低分样本自动进候选池，人工只做采纳与否。判据是「同一条失败模式一周内出现超过若干次」才进池，避免个案噪声污染用例集。

12.4 一条红线：在线信号不许直接改提示词

仓库里有一套自我演化机制，能根据在线信号让模型提出改进提案，并且有严格的白名单：只能改提示词、工具策略、记忆整理、验证器元数据、知识、程序这六类载体，载荷字段逐个白名单校验，验证器的四个文件禁止被改，证据内容不能当指令用。这套防线设计得相当克制，方向是对的。

但它目前只到提案为止：晋升接口把提案塞进内存字典再写一行日志，全仓没有代码去读这个日志并应用；回归门收的是一个布尔值，谁算这个布尔由调用方决定，它自己并不跑评测；灰度百分比只是一个数字。也就是说，这套机制现在是一个提案生成器，不是演化系统。

正确的闭环顺序是五步，缺一步都不能开自动化：在线信号 → 候选用例 →

离线回归 → 提案 → 灰度晋升。其中第二、三步是现在缺的，也是第 10 章要做的事。在此之前，正确姿势是让 monitor agent 只做「发现与归因」，把改动交给人——这比让它自动改提示词安全一个数量级，因为提示词改动的影响面是全局的，而它的效果只能在离线回归里看出来。

13 工具调用与检索失败的优化

这一类问题在实际系统里占比最高，也最容易被笼统地归为「模型不行」。实际上工具调用失败有明确的五种形态，每一种的上层动作都不同；检索不全也有明确的三层兜底，每一层的成本差一个数量级。把它们分开处理，比换一个更大的模型有效得多。

13.1 五类失败与各自的唯一动作

表 13-1 五类失败、现状与动作

类型	判据	现状	动作
工具不存在	调用名不在注册表	返回一句自然语言提示，无结构	改选工具，查能力注册表
参数不合法	schema 校验失败或服务端拒绝	无入参校验，直接透传	按错误信息修正一次
调用超时	超过单次预算	不重试，但也不归类	上报，计入熔断计数
结果为空	检索返回零条或全低于阈值	直接返回固定拒答文案	进入三层兜底
结果不可用	锚定分数低或引用不可验	只记警告，照常交付	补证据重答一次

「结果为空」这一行有一个具体的语义问题：现在的固定文案把两种完全不同的情况合并成了一句话——库里确实没有，和库里有但没检索到。前者应当直接说没有，后者应当说这次没找到、可以换问法或稍后重试，并且要记一次检索失败。把两者说成同一句话，用户会以为系统能力不足，运营也看不到真实的检索失败率。

「结果不可用」这一行的现状是默认只观察不拦截，模式开关有三档，默认是

观察。这意味着低质量答案照常交付。改成拦截要谨慎——拦截的代价是可用性，而误拦比不拦更伤害体验。建议的动作不是拦截，而是补证据重答一次：把未被证据支撑的句子取出来，带着它们回检索一次，仍不支撑再降级为警告。这条路径的成本是一次检索，远低于整段拒答。

13.2 检索不全的三层兜底

表 13-2 三层兜底的成本与触发条件

层	做法	触发条件	额外成本
一、改写重检	同义替换、上位词、去掉过窄限定后重检	空结果，或首条分数低于阈值	一次改写加一次检索
二、通道降级	混合检索失败退稠密，稠密失败退稀疏	某一通道异常或全部低于阈值	一次单通道检索
三、明确失败	区分「库中没有」与「本次未检索到」，各给不同文案	前两层耗尽	无

第一层的现状是「有改写但没有条件触发」：改写只作为检索前置步骤跑，无论检索结果好坏都跑一次，而空结果时反而不重试。这个逻辑是拧着的——前置改写的收益是提高命中，空结果重写的收益是修复失败，后者更值得做。改法是把改写拆成两个时机：一次预改写（可选，看成本），一次条件重写（空结果或低分必触发，且只做一次）。

第二层的现状是有级联但闸门关闭：级联逻辑在，最低分阈值配成零，等于永不触发。打开要同时给两个值——最低分阈值按分位数取（例如历史命中分数的低十分位），以及最多升级几级。阈值不按分位数取会踩同一个坑：不同语料的分数量纲不同，拍一个绝对值在一批文档上刚好，换一批就全触发或全不触发。

两层都失败之后的文案措辞值得较真。「未检索到相关内容，无法基于文档回答」这句话同时否定了两件事：库里没有，以及我的检索能力。正确说法要拆开，而且要把「换一种问法」这个可行动的提示给出来。这不是体验问题，是归因问题——一句话把系统故障说成知识缺失，线上就永远发现不了检索链路坏了。

13.3 重排与截断：两处正在悄悄丢分的实现

重排现在是替换式的：取前二十个候选送进重排模型，拿回来的结果直接覆盖原序列，第二十名之后的候选被静默丢弃，且重排分数不回写。这带来三个后果：一是下游读到的检索分数与真实顺序矛盾（顺序变了，分数没变）；二是动态跳过的判定、级联的分级归一、块对象上的分数三个地方读到的都是过期值；三是被丢掉的尾部候选本来是花了检索成本取回来的。改法是分数插值：最终分取重排归一化分与融合归一化分的加权，尾部候选按名次降级保留而不是丢弃，同时在阶段报告里记一个「丢弃条数」，让这个动作可见。

截断的问题有两个层面。配置层面，它按行数与字符数双限（一百行、八千字符），没有 token 口径——八千个汉字对三十二 K 上下文的模型接近四分之一预算，对更小的模型直接击穿。实现层面，截断器本身做得不错：有分页游标、有截断提示、提示会并进返回值，模型能知道自己看到的不全。真正的问题是它没接在主聊天链路上——那条链路的五个工具是裸函数，结果直接返回，只有文档条数的裁剪，没有字符上限。所以「结果太长」这类故障在主链路上是完全没有防护的。

两处一起改的顺序是：先把主链路工具接进执行器与截断器（这是接线，不改逻辑），再加 token 口径（复用上下文引擎已有的中文估算），最后才考虑按工具声明各自的 token 上限。顺序反了会得到一个「有精细限额但根本没生效」的系统。

13.4 自愈循环的边界

重试必须有硬边界，而现在的边界划得不对。步数上限是二十五，但模型步与工具步各计一次，实际可用的工具轮只有十二轮左右；且这个上限是全局的，无法约束「同一个工具被反复调用」和「单轮里并发几十个调用」这两种真实的失控形态。那个名字看起来像轮次门禁的模块其实不是门禁，它是并发派发前的授权检查

与危险工具串行化判定，跟轮次无关，名字会误导后续维护者。

要补三个独立的口径：单轮最多几个工具调用、单个工具在同一任务里最多调几次、总工具轮数上限。三个都要有，只给总数会被单个工具吃光。另外加一条一票否决：同一工具用相同参数连续失败两次，直接放弃并上报，不再进入下一轮。这条规则的价值是切断「模型反复试同一个不 work 的调用」这个最常见的死循环，而这个循环靠总步数上限要烧掉十几轮才停。

自愈的另一个边界是：修参数重试只允许一次，且必须基于错误信息里的具体字段。允许无限修参数等于把参数校验交给模型赌运气，实际表现是 token 烧光而问题没解决。这一条和第 8 章的错误分类是同一件事的两面——那边给出「哪类错误可以改参数重试」，这边规定「改几次」。

14 测量基础设施

前面三章要的数字都依赖同一套底座。这一章只做压缩陈述，说明现有底盘如何支撑，不展开评测方法论。

14.1 数据划分

表 14-1 三集划分与用途

集合	用途	约束
训练集	挑预算档位、示例保留集合	可反复使用，允许调参
验证集	确认拐点、复核回退条件	每轮只跑一次冻结配置
测试集	最终读数，只跑一次	不参与任何选择，否则读数作废

14.2 三层拆分与回放

表 14-2 三层评测的耗时与用途

层	覆盖	单轮耗时	用途
L1 路由	只做意图分类	秒级	每次改提示词都跑，进持续集成
L2 检索	检索与融合排序	分钟级	改索引或预算时跑
L3 端到端	完整链路	小时级	只在候选版本上跑，走回放

回放是这套拆分的成本前提：录制一次之后，重复执行不产生新的调用，改评分函数与阈值都不花调用费。录制只在语料或提示词大改时重做。

14.3 被测模型的冻结

四要素写进快照文件并在评测启动时校验：模型标识与权重版本（本地部署要写明量化格式与运行时版本）、温度、随机种子、上下文与最大输出长度。不一致直接失败，不静默换档。原因很直接：本方案的结论都是相对增量，换一个档位，增量会变，数字就不再有意义。

14.4 门禁与基线

门禁四态——通过、失败、未武装、未强制——缺基线时硬失败，建档只走唯一通道。本次需要在基线里增加版本列，使「这一版提示词」成为可比较的主体，而不是每版都覆盖上一版的读数。

15 施工顺序与工时

顺序按依赖排，不按重要性排。前四步是测量与版本化，它们不改产品逻辑，但决定后面每一步的迭代成本。

表 15-1 施工计划

序	动作	落点	产出	预估
1	时延分段补齐与分桶	计时账本补四段；分位数按路由类型分组	六段账本与分桶 P95	0.5 天
2	提示词注册表	四处常量收敛为单表，导入期校验唯一性	可引用的单源提示词	1 天
3	版本埋点与用量标签	用量记录增加版本标签，轨迹写入版本号	token 可按节点与版本归因	0.5 天
4	预算接线	版本预算接入上下文引擎的既有预算参数	预算固定不随内容漂移	0.5 天
5	回放通道	录制装饰器与评测入口的回放开关	零调用重跑	0.5 天
6	基线录制与门禁武装	录基线 → 建档 → 恢复强制	第一条带版本列的基线	0.5 天
7	低风险两条压降	预算收紧、结构化输出	第一轮 token 降幅与完成率对照	1 天
8	示例裁剪逐条对照	每条示例单独去掉跑一轮	最小可用示例集	1 天
9	时延三条措施	规则直判、同问缓存、超时收紧	分桶 P95 对照	1 天
10	完成率口径落定	剔除项写进评测入口，三数并列输出	可比完成率	0.5 天
11	路由置信度与决策日志	提示词放开置信度与依据、结构校验必填、五字段落轨迹	路由决策可复盘	1 天
12	路由分层评测集	按类别重标，每类不少于三十条，另设边界样本	有区分度的路由指标	1 天
13	MCP 生产接线	服务注册表、按需启动、出口过截断与错误分类与熔断	MCP 真正被调用	1.5 天
14	记忆写路径与召回乘子	会话结束入队摘要；频次与最近命中进打分；注入点前移	跨会话记忆可用	1 天
15	轨迹结构合并与工具步补全	两套轨迹收敛为一套；补工具步耗时与载荷摘要	可复现的轨迹	1 天
16	会话级评测集	四类用例各含负例，从真实会话改造	多轮与记忆的尺子	1.5 天
17	能力注册表与候选打分	注册表落地；四因子打分；串行备选与阈值	路由可配置而非硬编码	1.5 天

序	动作	落点	产出	预估
18	在线闭环与门禁恢复	补失败计数与记忆类指标；低分样本自动进候选池；独立集成作业	闭环与防回退	1 天

前十个步骤合计约七个工作日，构成第一部分的主线；后八个步骤合计约九个半工作日，对应第二部分的七项能力。两部分之间是依赖关系而不是并列关系：第 3、5、6 步产出的版本标签、回放通道与基线，是第 11 至 18 步所有结论的前提——没有它们，第二部分只能交付「功能已上线」，交不出「效果如何」。反过来，第 15 步的轨迹结构合并是第 10、12 章评测能否落地的前置，排在记忆与路由之后做会导致前两步的埋点白做，因此若工期紧张，优先把第 15 步提前到第 11 步之前。

第 7 至 9 步每步结束都要出一张同表对照，不允许攒到最后一起看。第 13 步之后 MCP 才有流量，此前关于它的任何性能论述都只能是设计推演。

16 有效性威胁与边界

这一节列出会让上面结论失效的条件。它们不是免责声明，而是决定每个数字能说多重的话。

1. **样本规模**。测试集只有十几条，完成率的最小可检测差异接近二十个点。因此完成率只用于「不降」的判定，不用于「提升」的宣称。
2. **单一模型档位**。所有读数绑定本地模型的量化档位，换模型或换量化格式必须整表重测。弱模型的压缩空间更大，token 降幅不可外推到强模型。
3. **回放的局限**。回放固定了模型输出，因此无法反映模型侧的非确定性；重复执行的一致性必须在真实生成下单独测一次，不能只看回放。
4. **分位数口径切换**。切换日之前的读数与当前不同尺，任何跨口径对比都无效，做对比前必须重跑基线。

5. **缓存的前提**。同问缓存的收益取决于重复问句比例，这个比例在本项目的真实流量里尚未统计，收益可能是零。
6. **样本来源**。金标由单人写定，未算标注一致性；完成率的绝对值受标注质量限制，相对变化更可信。
7. **第二部分的能力指标多为计数量**。工具调用成功率、记忆命中率、空结果率这类指标对样本量友好，但它们只能说明「链路是否通」，不能说明「效果好不好」。把覆盖率写成质量提升，是这一部分最容易犯的错。
8. **多智能体部分的结论依赖开关状态**。隔离、契约、总线三个开关默认关闭，默认状态下测得的「多智能体表现」实际上是单智能体加自审。任何引用这组数字的场合，都必须同时声明开关状态，否则读数不可解释。
9. **在线信号存在幸存者偏差**。低分样本来自用户反馈与自动校验，而沉默的不满不产生反馈。因此「低分样本转用例」只能覆盖可观测的失败，覆盖不了「用户放弃使用」这一类最严重的失败。这一项的缺口无法用现有手段补齐，只能如实注明。

附录 A 未作为主线的部分

以下能力已经建好，但本方案不作为主线，理由写在这里以免被误读为遗漏。

路由准确率与多智能体消融：前者在现有样本规模下最小可检测差异接近二十八个点，任何个位数改进都不可分辨；后者依赖端到端真跑，成本高且结论与成本、时延两类指标不共享分母。两者作为工程资产保留，改动评测基础设施时仍会跑，但不进入本方案的结论表。

过程层指标（工具调用匹配、计划正确性、首错定位、步数效率）同样保留在评测入口里。它们对本方案的唯一作用是解释 token 变化的来源——例如步数效率下降时，token 上升是任务变难还是提示词变啰嗦，需要靠它区分。这类指标只作诊断，不作为结论。